LINFO2144 - TP secure communication channels

1 Symmetric channel constructions

We consider the communication channels described by the following algorithms. Identify which ones are correct, and among those, which ones are secure. Discuss which security properties are satisfied (and argue why), and describe attacks for security properties that are not satisfied.

In the algorithms,

- $(\mathrm{Enc}, \mathrm{Dec})$ is an AEAD with key space $\mathcal{K}$, F is a KDF with output in $\mathcal{K}$;
- the role $R \in \{0, 1\}$ is different for each party, the role of the party executing a procedure is passed as an argument to all procedures;
- exponents denote the party who computes/knows the value of variables in the algorithms, variables initialized in Init are part of a state that can be accessed and modified by all procedures;
- Init is called once for each party, with a random key $k \in \mathcal{K}$ (the same for both parties). Then, Send and Recv are called for each message to be sent/received.

Algorithm 1

procedure INIT(k, R)
  $k_s^R \leftarrow F(k, R)$.
  $k_r^R \leftarrow F(k, 1 - R)$
  $n^R \leftarrow 0$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_{k_s^R}(n^R, m)$
  $n^R \leftarrow n^R + 1$
  return c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_{k_r^R}(n^R, c)$
  if $m = \perp$ **then**
    return $\perp$
  end if
  $n^R \leftarrow n^R + 1$
  return m
end procedure

Algorithm 2

procedure INIT(k, R)
  $k_s^R \leftarrow F(k, R)$.
  $k_r^R \leftarrow F(k, 1 - R)$
  $n_s^R \leftarrow 0$
  $n_r^R \leftarrow 0$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_{k_s^R}(n_s^R, m)$
  $n_s^R \leftarrow n_s^R + 1$
  return c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_{k_r^R}(n_r^R, c)$
  if $m = \perp$ **then**
    return $\perp$
  end if
  $n_r^R \leftarrow n_r^R + 1$
  return m
end procedure

Algorithm 3

procedure INIT(k, R)
  $k_s^R \leftarrow F(k, R)$.
  $k_r^R \leftarrow F(k, 1 - R)$
  $n_s^R \leftarrow 0$
end procedure
procedure SEND(m, R)
  $c \leftarrow (n_s^R, \mathrm{Enc}_{k_s^R}(n_s^R, m))$
  $n_s^R \leftarrow n_s^R + 1$
  return c
end procedure
procedure RECV(c, R)
  $(n, c) \leftarrow c$
  $m \leftarrow \mathrm{Dec}_{k_r^R}(n, c)$
  if $m = \perp$ **then**
    return $\perp$
  end if
  return m
end procedure

Algorithm 4

procedure INIT(k, R)
  $n_s^R \leftarrow 0$
  $n_r^R \leftarrow 0$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_k(n_s^R, m)$
  $n_s^R \leftarrow n_s^R + 1$ **return** c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_k(n_r^R, c)$
  if $m = \perp$ **then return** $\perp$
  end if
  $n_r^R \leftarrow n_r^R + 1$
  return m
end procedure

Algorithm 5

procedure INIT(k, R)
  $n_s^R \leftarrow R$
  $n_r^R \leftarrow 1 - R$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_k(n_s^R, m)$
  $n_s^R \leftarrow n_s^R + 2$ **return** c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_k(n_r^R, c)$
  if $m = \perp$ **then return** $\perp$
  end if
  $n_r^R \leftarrow n_r^R + 2$ **return** m
end procedure

Algorithm 6

procedure INIT(k, R)
  $n^R \leftarrow 0$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_k(n^R, m)$
  $n^R \leftarrow n^R + 1$ **return** c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_k(n_r^R, m)$
  if $m = \perp$ **then return** $\perp$
  end if
  $n^R \leftarrow n^R + 1$ **return** m
end procedure

2 Confidentiality beyond standard definition

TLS traffic sniffing. Assume you are monitoring the IP traffic between a user's device and the server of a messaging app, which is encrypted with TLS.

You make the following observations: At regular intervals, a couple tiny packets are sent. At some point, one of the packets sent by the device is a bit (e.g. 200 bytes) larger than usual.

What can you conclude? In particular, assume you are running an online advertising business, and you just displayed an ad to that user.

What are the communication protocol's characteristics that are exploited in this attack? What can a protocol do in order to protect against such attacks?

Interactive SSH. When typing on a keyboard, the delay between consecutive keystrokes is highly dependent on the person typing and the characters being typed. Moreover, when using a remote shell over SSH, keystrokes need to be send with low latency, otherwise the usability is degraded. Assume a naive SSH client implementation, how is this a security issue? What is a typical sensitive piece of data interactively sent over SSH?

Analyze the countermeasure introduced by OpenSSH 9.5 using the release notes (`https://www.openssh.org/txt/release-9.5`) and the configuration

description (`ObscureKeystrokeTiming` in `man 5 ssh_config`).

What are the trade-off of the parameters?

3 Timing Side-Channel Attack

In this exercise, you will have to perform a timing side-channel attack to retrieve a pin code. Instead of doing a simple brute force, find a way to retrieve the pin code with fewer requests. For this exercise, we propose you to build a docker to have a "black box" to query. You can also make a virtual environment or install all Python modules needed on your laptop. All the files are available in the folder `1-Timing_Attack`. For the docker, run the following commands:

```
docker build . -t timing_attack
docker run -d --network host timing_attack
```

The website is available at `http://localhost:5000/`.

1. As a first step, try to guess the length of the PIN, without guessing the PIN itself. Looking at the `server.py` code, how will the verification time for a PIN of the correct length compare to the time for an incorrect length?

2. As a second step, try to guess the PIN.
   - How many attempts will you need if you brute-force the PIN?
   - How many attempts will you need if you guess digits one-by-one?
   - How can you find the first digit of the PIN?
   - How can extend this attack to all digits?

3. Once your attack is successful, change the pin code and try again with your solution. It should still work even if the length changes.

4 SSH with compression security

In this exercise[1], you will use SSH with compression. All the files needed for this exercise are in the folder `2-ssh_compression`. You have a client and a server. The client compresses the data before encrypting it. As an attacker, in this scenario, you are able

[1] This exercise has been adapted from a lab of the 6.1600 course given at MIT (`https://61600-labs.csail.mit.edu/lab3.html`).

to ask the client to add an evil string that will be concatenated with a secret that the client sends to the server. So the client will send `evil+secret` to the server. The `secret` is composed of five capital cities chosen at random in a list and serialized into a JSON object. For example:

```
{
"city0": "Victoria, Seychelles",
"city1": "Seoul, South Korea",
"city2": "Paris, France",
"city3": "Vatican City, Vatican City",
"city4": "Paramaribo, Suriname",
}
```

As a first step, let us look at the behavior of compression algorithms. Using `zlib` compression (`zlib.compress` in Python), compare the length of a string of $2n$ distinct characters versus the concatenation of 2 identical strings composed of n characters (e.g. the alphabet vs twice half the alphabet). Can you extend your observations to the concatenation of `evil` and `secret`, for some well-chosen `evil`? How can you mount an attack using a compressed message length oracle?

Next, your goal is to modify `attack.py` to perform an attack and find the secret. You will need to call the function `(bytes_out, bytes_in) = client_fn(<evil_string>)`. This function causes the client to send a message to the server. It returns the number of bytes sent to the server (`bytes_out`) and the number of bytes received from the server (`bytes_in`).

For this exercise, we propose you to build a docker. You can also make a virtual environment or install all Python modules needed on your laptop. For the Docker version, you can build the Docker with `docker build . -t ssh_compression`. To launch the Docker, you can use the command:

```
docker run --mount type=bind,src=./attack.py,dst=/app/attack.py ssh_compression
```

Finally, in a new terminal, connect to the docker with:

```
docker exec -it <docker_name> /bin/bash
```

Execute the command `python3 main.py` to test your attack. You can change `attack.py` without rebuilding the Docker.

Finally, can you imagine an hypothetical practical setting where such an attack could be applicable (you may assume a different protocol, such as HTTPS)? Which adversary could be interested in recovering a list of cities (or another list of words)? How could an adversary have control of an `evil string`?

5 Performance comparison

In this exercise, you will have a look at the performance of various hashing and encryption algorithms. Use the VM if you do not have OpenSSL installed on your laptop.

To compare the performance of hashing and encryption algorithms, we will use `OpenSSL`. The VM runs with the version `1.1.1d`. The current stable version is `3.4.1` with many vulnerabilities fixed.

The command `openssl help` shows the different standard, message digest and cipher commands. The command `openssl speed <algorithm>` performs a speed test.

In the first instance, we are interested in the message digest `SHA` (Secure Hash Algorithm). Draw a graph comparing the performance of the different versions (SHA2 in its variants `sha256` and `sha512`, SHA3 in the variant `sha3-256`). Discuss your results. For each algorithm, explain why or why not you would use it. Give a short example of the context where you could use it.

In a second step, we are interested in authenticated encryption algorithms and more specifically the AES (Advanced Encryption Standard) encryption with GCM (Galois Counter Mode). As previously, compare the `aes-128-gcm` and `aes-256-gcm` algorithms and explain in which context you should use which one.

Beyond the comparison, in which practical situations are these functions likely to be (or not be) performance bottlenecks?

Run benchmarks for asymetric cryptography algorithms, e.g. using Curve 25519 for ECDHE (X25519) and for signature (Ed25519). Would those be bottelnecks in practice?

Cryptographic engineering is undergoing a transition towards new algorithms that resist cryptanalysis by quantum computers (so-called "post-quantum cryptography"). Which of the algorithms discussed above are PQ-secure? How to the PQ algorithms recently-standardized by the NIST compare to non-PQ algorithms in terms of performance? (Performance is not only about computation time, but also about the size of the data.) Wikipedia can be a good starting point for your investigations [2].

[2] `https://en.wikipedia.org/wiki/Post-quantum_cryptography`